Scalable Mass Crowd Simulation with Multi-Agent Behavior
Stanford CS348K: Final Project Report

Alex Liu (alexlpc), Daniel Song (djsong)

May 28, 2026

1 Background

Creating a real-time simulation of large populations of independently moving agents is computationally expensive and difficult. Nevertheless, many games, including real-time strategy (RTS), city-building, colony survival, and sandbox titles, depend on the smooth simulation of large crowds of autonomous entities. In this project, we aim to build a real-time multi-agent simulation in Unity in which a large number of Non-player characters (NPCs) navigate and perform simple tasks within a shared game environment. Our goal is to maximize the number of agents that can be simulated while maintaining an interactive frame rate. Achieving this requires balancing limited computational resources with the need to update the simulation efficiently each frame to preserve responsiveness and interactivity. In other words, we seek to minimize the frame time while maximizing the number of agents in the simulation.

To explore this optimization problem, we begin with a naive baseline in which each NPC independently updates its behavior every frame. From there, we incrementally develop more efficient system designs using techniques such as GPU instancing, spatial partitioning, entity component systems (ECS), and behavior level-of-detail (LOD). The inputs to the system include the scene layout, navigation mesh, NPC count, and configuration parameters associated with each optimization technique. The outputs consist of real-time NPC trajectories and behaviors, along with performance metrics such as frame time and simulation throughput.

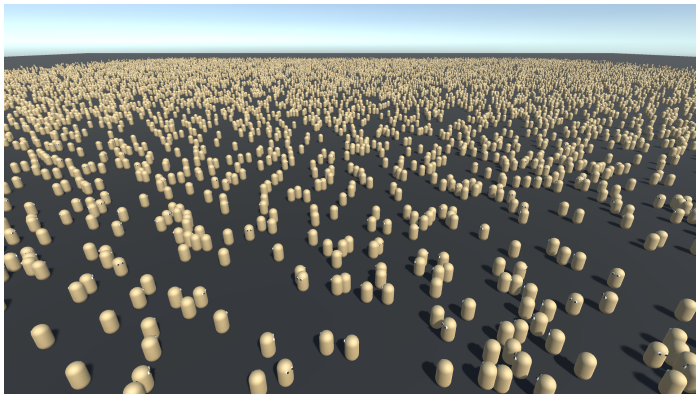

Figure 1: An example of simulating 20,000 agents using our final system running at an average of about 45 frames per second on a mid-high-end desktop computer using ECS (non-LOD) mode. The device specifications are listed in detail in section 4.2.

2 Project Setup

To ensure a consistent setting for the fair evaluation of the optimization strategies, we consider a simplified scenario consisting of a large, flat playing field populated by geometrically simple agents.

Each agent is represented as a colored capsule with eyes made up of two spheres per eye, representing the eyeball and pupil, which indicate the agent's orientation. The task of each agent is to navigate to a randomly assigned to a uniformly sampled target position on the field while avoiding other agents in the way (mimicking how characters would move in an RTS or city-building game). Whenever an agent reaches the target position, a global task completion counter is incremented and a new target position is sampled.

3 Approach

3.1 Baseline

We began with a naive implementation of the system by using Unity's built-in navigation system. We instantiate N agents in random starting positions across the playing field and run the simulation. Unity's Navigation system uses a pre-baked navigation mesh, defining the walkable areas where an agent can stand or move around in a game scene. These navigation meshes represent the surface with convex polygons, which are used as nodes for the pathfinding algorithm. Specifically, Unity uses the A* algorithm to search for paths and performs a simple funnel algorithm to tighten the path to the target location. The built-in implementation performs all path calculations on the main thread. Once a path is calculated, Unity runs the simulation across multiple threads of worker cores while using reciprocal velocity obstacles (RVO) to predict and prevent collisions between agents or other moving obstacles.

3.2 GPU Instancing

With the rendering of the agents being the primary bottleneck of the baseline system, we implement spatial hashing and GPU instancing. This is a common technique used in computer graphics to reduce the number of expensive draw calls to the GPU by batching the many copies of the same mesh into a single draw call. To support an arbitrary number of agents, we partition the space into an evenly spaced grid of cells and issue one instanced draw call per cell (the Unity API has a maximum limit of 1023 instances for each instanced draw call). This spatial partitioning scheme will also be useful for the entity component system later on.

This technique assumes that all agents use an identical underlying mesh. However, as large-scale crowd simulation systems in games typically employ duplicated geometry for agents, we consider this as a reasonable compromise.

3.3 Entity Component System

Another common technique used in the game industry for large-scale simulations is the Entity Component System (ECS). ECS is a data-oriented architectural pattern that emphasizes composition over inheritance-based object-oriented design. In ECS, each entity is typically represented by a unique identifier. Entities can also be assigned components, which are lightweight data structures containing simple values. Systems define behavior by iterating over entities that match specific component compositions, processing only the relevant data for each operation. This design improves data locality, making the system more cache-friendly and better suited for parallel execution. Unity has an ECS package known as Unity Data Oriented Technology Stack (DOTS), which we use to implement our system.

One limitation of this approach, however, is that we can no longer use the built-in navigation system used in the baseline approach (as the navigation system is not thread-safe). To circumvent this, we use our own, simplified navigation algorithm. Each agent has a component that defines its target location and velocity vector. Whenever a new target position is sampled, the agent adjusts its velocity direction toward that target. At the same time, it queries the spatial grid described in Section 3.2 to identify nearby agents. For each neighboring agent, a repulsion vector is computed and scaled by the inverse cube of the separation distance. These repulsion vectors are then added to the desired movement direction, enabling local collision avoidance. Despite its simplicity, this approach can be easily extended to support waypoint-based navigation generated by a pathfinding

algorithm to avoid static obstacles. Using Unity's DOTS API, this system is distributed across multiple worker threads, enabling a multithreaded navigation solution. This contrasts with the baseline system, which performs the majority of its navigation processing on the main thread. In practice, we also use this style of navigation for the GPU instancing mode.

3.4 Behavioral Level-of-Detail

This approach is based on the assumption that, in large-scale simulations, entities farther from the point of interest require less behavioral fidelity. As a result, the quality and detail of task execution can be reduced for distant entities without significantly affecting the overall simulation. We implement this system by reducing the update frequency of agents' behavior as their distance from the point of interest increases.

Specifically, we define three distance thresholds that classify agents according to their proximity to the point of interest, which is the center of the field in our experiments. Each class is assigned a different update frequency, with more distant agents being updated less frequently. For each class $c \in \{0, 1, 2, 3\}$ (with $c = 0$ being the class closest to the point of interest), the agent's behavior is updated once every 2^{c-1} ticks.

This behavioral update is distinct from the agents' general update step. The behavioral update involves recomputing the target velocity based on path-following objectives and the relatively expensive local avoidance system. Agent movement is still updated every frame, regardless of class, to ensure smooth and continuous motion throughout the simulation.

3.5 Learned Policy

As an additional experimental approach, we also implement a simple learned policy to drive the agent's behavior. The current setup for the system is to distribute the behavioral update of each agent across multiple worker threads. However, the ideal approach would be to utilize the parallelism of the GPU to perform the behavioral updates of all the agents. We avoid using compute shaders to retain full CPU ownership of agent data, resulting in a more flexible and developer-friendly architecture for future game-oriented extensions. An alternative solution is to use a small multilayer perceptron (MLP) to predict the velocity updates in large batches. One limitation of this approach is that the MLP does not provide the same collision-avoidance guarantees as the original local avoidance system. As a result, agents may occasionally interpenetrate.

We use an MLP that takes as input the target offset, current velocity, offsets to the nearest neighboring agents, and the distance to the field boundary. The network outputs a predicted velocity for the agent at the next update step. The MLP is a lightweight model consisting of a single hidden layer with 64 neurons and a ReLU activation function, which is trained on data generated by our simulation system. During inference, we use a batch size of 4,096, allowing the behaviors of up to 4,096 agents to be updated simultaneously.

4 Evaluation and Results

4.1 Goals

The primary goal of this project is to maximize the number of well-behaved agents that can be simulated in real time on a fixed hardware platform. We define success as achieving a significant increase in the maximum supported agent count while maintaining an interactive frame rate of at least 30 FPS, without substantially degrading agent behavioral quality.

We propose that a combination of GPU instancing, data-oriented simulation through ECS, and behavioral level-of-detail (LOD) can substantially increase the maximum supported agent count relative to a naive, baseline Unity implementation without significantly degrading agent behavior

quality.

To evaluate this, we compare the following system configurations:

1. baseline Unity NavMesh implementation
2. GPU Instancing & Spatial Hashing
3. ECS-based simulation
4. ECS & Behavioral LOD
5. Learned Policy variant

For each approach, we measure average frame rate, frame time, and task completion rates across increasing agent counts.

4.2 Experimental Setup

All experiments were performed on the same machine to ensure fair comparisons. Specifically, we use a custom machine equipped with an Intel Core i5-13600KF CPU and an Nvidia RTX 3090 Ti GPU. Our system is built on Unity version 2021.3.45f2.

As described in Section 2, agents repeatedly navigate toward randomly generated target positions while avoiding nearby agents. Whenever an agent reaches its target, a new target is assigned, and a global completion counter is incremented. We avoid more complex tasks, such as object manipulation or combat, to prevent task depletion and changes in the number of active agents during measurement. This ensures a consistent workload and allows throughput to be measured reliably throughout the experiment. However, the system is designed so that such tasks can be easily implemented if required in a practical game application.

The field is scaled to maintain a consistent ratio between the total area occupied by agents and the overall field area. This prevents larger agent populations from experiencing artificially lower task completion rates due to increased congestion, ensuring that the level of congestion remains approximately constant across different agent counts.

Since each agent occupies a circular area on the field (corresponding to the cross-section of the capsule-shaped agent), the dimensions of the square field are determined by the desired occupancy ratio. Specifically, the length of each side of the field is given by $\sqrt{\frac{N\pi r^2}{P}}$, where N is the total number of agents, r is the radius of an agent's circular footprint, and P is the target occupancy fraction of the field area. This ensures that the proportion of occupied space remains constant as the number of agents varies. In our project, we use a constant $P = 0.05$, indicating a 5% occupancy ratio.

We evaluate each algorithm at agent counts of 1,000, 2,000, 5,000, 10,000, 15,000, and 20,000. Metrics are collected automatically through our logging framework and aggregated over the duration of each experiment. Each simulation is run for a 2-second warm-up period, followed by a 15-second measurement window during which average metrics are recorded. To reduce variance in GPU workload and rendering time, all experiments are conducted using a fixed camera position. We also use the built-in profiler to analyze hardware resource utilization and the execution time distribution across system processes.

4.3 Performance

The baseline system performance struggles as the agent count increases. The majority of this decline is caused by the substantial GPU workload required to render the agents' polygons, as well as the CPU overhead of executing the pathfinding calculations for all agents on the main thread. At a scale of 10,000 agents, rendering constituted approximately 41.9% (17.09 ms) of the total frame time, making it the largest performance bottleneck. The navigation system accounted for a further 27.9% (11.36 ms), with the remainder spent on general engine operations such as physics simulation,

garbage collection, and other maintenance tasks.

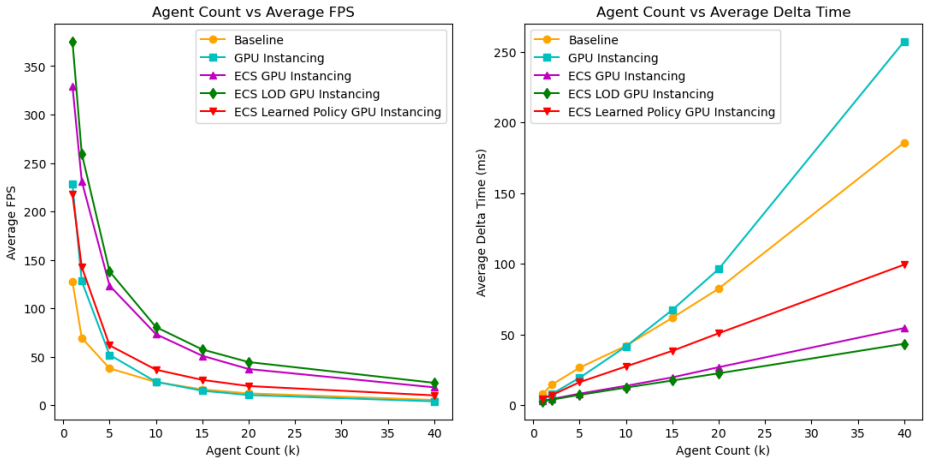

Figure 2: A comparison of the FPS and delta time performances across system types. (left) The average FPS generally improves across systems, with the exception of GPU instancing at higher agent counts and the learned policy approach. (right) The delta time graph exhibits mostly linear growth across all systems, mirroring the relative performance trends observed in the FPS results.

Interestingly, GPU instancing performs better at lower agent counts (below 10,000 agents), but becomes less efficient as the number of agents increases. This behavior is likely due to the cost of spatial hashing eventually dominating the overall computation time. Because the CPU must serially assign every agent to a spatial hash cell each frame, the hashing workload grows significantly with agent count and field size. In addition, the fixed cell width of 2.5 units across all simulations means that larger simulation fields introduce additional overhead in the batching process, contributing to the observed decline in performance. This is supported by profiler results, which show that while the rendering contribution per frame drops significantly to 6.5%, scripting logic and behavior updates account for 62.8% of frame time at 10,000 agents. This effect becomes even more pronounced at 40,000 agents, where rendering accounts for only 3.6% of frame time, while scripts increase to 78.4%.

As for the ECS implementations, Figure 2 and Table 1 show that the cache-efficient approach, combined with the parallel execution of behavioral updates, leads to a substantial performance improvement compared to the previous systems. The ECS and ECS LOD methods achieve an approximately consistent 3x increase in FPS across all agent counts, demonstrating that the ECS paradigm is highly effective for large-scale crowd simulations involving basic navigation tasks. The behavioral LOD also consistently performs slightly better than the standard ECS approach without LOD, indicating that staggered updates for distant entities help reduce CPU load, likely more evident because distant agents no longer perform the nested neighbor calculations every frame. This is confirmed by the reduction in the ECS agent simulation's contribution to frame time, which decreased from 18.8% (9.65 ms) to 3.1% (1.33 ms) when profiling a simulation with 40,000 agents.

On the contrary, the experimental learned approach performed significantly worse than expected. We suspect this is due to the simplicity of the behavioral updates and tasks to be able to benefit from the parallelism and learned execution strategy, which is better suited to more complex instruction sets. In the current implementation, the behavioral update step only involves moving toward a target location while applying repulsive forces from nearby agents. Furthermore, unpacking the generated output is performed sequentially, introducing additional overhead.

4.4 Behavioral Quality

Quantifying behavioral quality is inherently challenging. Therefore, we evaluate agent behavior through both visual inspection and a coarse quantitative measure based on task completion rate.

System	1k			2k			5k		
	FPS	DT (ms)	TCR	FPS	DT (ms)	TCR	FPS	DT (ms)	TCR
Baseline	127.5	7.85	0.0281	69.4	14.40	0.0132	37.9	26.50	0.0023
GPU Instancing	228.2	4.38	0.0247	128.0	7.83	0.0131	51.84	19.29	0.0062
ECS	329.5	3.04	0.0252	231.3	4.32	0.0136	123.3	8.11	0.0061
ECS LOD	375.4	2.66	0.0218	259.7	3.85	0.0118	138.0	7.25	0.0051
Learned Policy	217.7	4.59	0.0215	142.6	7.01	0.0118	61.69	16.25	0.0057

System	10k			15k			20k		
	FPS	DT (ms)	TCR	FPS	DT (ms)	TCR	FPS	DT (ms)	TCR
Baseline	23.85	41.94	0.0007	16.19	61.78	0.0005	12.13	82.42	0.0003
GPU Instancing	24.09	41.52	0.0033	14.85	67.37	0.0025	10.40	96.29	0.0019
ECS	73.28	13.65	0.0032	50.95	19.63	0.0024	37.26	26.84	0.0019
ECS LOD	80.62	12.40	0.0027	57.45	17.41	0.0017	44.43	22.51	0.0014
Learned Policy	36.59	27.36	0.0030	26.06	38.40	0.0022	19.67	50.86	0.0017

Table 1: Performance comparison across systems and agent counts. The 40k simulation results are omitted due to spacing constraints.

4.4.1 Visual Inspection

The baseline system, which uses the built-in navigation mesh and navigation agent components, appears to have an internal limit on the number of paths its pathfinding algorithm can generate per frame. As a result, many agents remain stationary until the system catches up with pending path calculations. This behavior becomes noticeable at around 2,000 agents, where nearly half of the agents remain stationary during the first few seconds of the simulation. At 5,000 agents, most agents appear stationary for the entire 15-second simulation, indicating that the pathfinding system cannot keep pace with the demand.

Visually, agents using the built-in navigation mesh system appear to adjust their trajectories preemptively to avoid collisions. They steer away from nearby agents while attempting to maintain their intended direction and speed, likely due to the predictive nature of the RVO algorithm. In more crowded scenarios, however, agents also reduce their speed and may come to a complete stop until the obstruction is cleared. The custom navigation algorithm, on the other hand, is purely reactive rather than predictive. Instead of anticipating potential collisions, agents respond only to nearby obstacles by applying a repulsion force that steers them away from surrounding agents.

The behavioral LOD system operates largely similarly to the other custom navigation algorithms. However, distant agents occasionally briefly interpenetrate in crowded regions. This is likely due to lower-frequency velocity updates, which cause these agents to respond more slowly than those updated at higher resolution.

The learned system operated nearly identically to the custom navigation systems, which is expected given that it was trained on them. Although rare, occasional interpenetrations were observed, though these would likely be mitigated with longer training and a larger dataset.

4.4.2 Quantitative Evaluation

To quantitatively compare behavior across all systems, we use per-agent task completion throughput as a relative measure of behavioral quality. This metric is calculated by dividing the number of completed tasks by the number of agents in the simulation and then dividing the result by the measurement duration in seconds. Figure 3 shows that, at low agent counts, the baseline system achieved the highest task completion rate, suggesting that the internal collision-avoidance algorithms enable a more efficient navigation. However, this relative performance quickly degrades as the agent count increases due to the internal limits mentioned in Section 4.4.1.

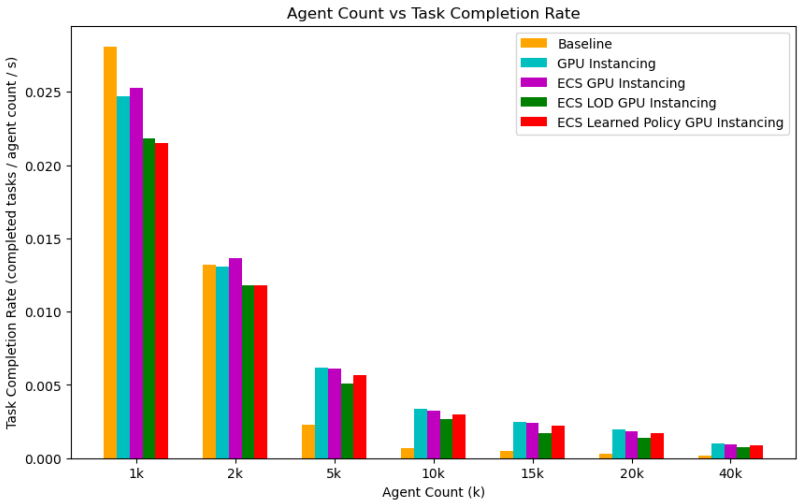

Figure 3: A comparison of the task completion rates normalized to the agent counts (completed tasks/total agent count/second) across all system implementations. The overall values decrease as field size increases with higher agent counts, since randomly sampled target locations tend to be farther from each agent's starting position.

In practice, we implement all other systems using the custom navigation approach described in Section 3.3 due to changes in the codebase for creating the training dataset for the learned approach. This leads to a minor drop in task completion rates compared to the baseline at lower, ideal agent counts. GPU instancing and the ECS approach produce similar results.

Interestingly, GPU instancing yields a slightly lower completion rate at low agent counts but a higher completion rate at larger agent counts, despite both systems using the same navigation algorithms. This may come down to differences in the simulation's delta time. Since the underlying kinematic integration scales with frame time, the significantly lower frame rates experienced by the GPU instancing system at higher agent counts result in larger discrete movement steps. This reduced temporal resolution may cause minor numerical inaccuracies in collision avoidance, occasionally allowing agents to "slip" through dense crowds, which may artificially inflate their task completion rate. This phenomenon was confirmed through visual inspection, where occasional agent interpenetrations were observed at low frame rates.

The use of the behavioral LOD system further reduces the task completion rate due to the lower update frequency of distant agents. However, this reduction appears less pronounced at higher agent counts, suggesting that behavioral LOD may be a reasonable trade-off for performance in large-scale simulations.

The learned policy approach performs very similarly to the LOD system and even surpasses it at high agent counts. However, its overall completion rate remains lower than the non-baseline, non-LOD approaches. We consider this a reasonable level of behavioral performance, particularly given that local collision avoidance is learned from parameters rather than explicitly defined mathematical rules, suggesting that the approach may be extensible to more complex tasks.

4.5 Discussion and Limitations

Our results indicate that transitioning to a data-oriented architectural pattern yields the most significant improvements. By utilizing an Entity Component System, we achieved a massive reduction in CPU overhead through improved data locality and multithreading. This approach proved to be the most robust solution for large-scale multi-agent simulations. When combined with Behavioral LOD, it supported 30,000 agents while improving the behavioral performance of the baseline approach at high agent counts. This represents a more than 4.6× increase in maximum supported agents, fully satisfying the primary project objective.

System	Max Agents
Baseline	6,500
GPU Instancing	8,500
ECS	25,500
ECS LOD	30,000
Learned Policy	7,000

Table 2: Maximum number of agents supported by each system while maintaining a stable 30 FPS over a 10-second interval. The table reveals that we were able to successfully satisfy our definition of success by increasing the maximum number of supported agents by over 4.6 times the baseline.

While other methods provided insight, they fell short at maximum scale. GPU instancing effectively eliminated rendering bottlenecks but ultimately shifted the bottleneck to the CPU due to the serial overhead of spatial hashing. Similarly, our experimental learned policy highlighted the difficulty of replacing explicit local avoidance algorithms with lightweight neural networks, particularly when strict collision resolution and multithreaded inference overhead are factored in. However, we hypothesize that the learned approach may become more advantageous for more complex decision-making tasks, compared to executing hardcoded instructions for each agent. A potential future direction is to process large spatial batches of agents as input, allowing the network to infer behavior from spatial context without relying on costly nested loops.

Currently, our experiments are conducted on simple crowd simulation tasks. Future work may explore more complex tasks and more sophisticated navigation environments, such as those involving procedurally generated paths. However, such environments may be more difficult to evaluate quantitatively due to increased variability across runs and population sizes.

5 Team Responsibilities

Alex and Daniel collaborated closely in person throughout the project and contributed comparably across all components. While Alex spent more time on certain implementation tasks and Daniel spent more time on evaluation and experimentation, both were involved in design, debugging, and interpretation of results, and each area was developed through joint discussion and iterative feedback.

References

[1] "Ai navigation 2.0.12: Nav inner workings," unity Documentation. [Online]. Available: https://docs.unity3d.com/Packages/com.unity.ai.navigation@2.0/manual/NavInnerWorkings.html

[2] "Ecs for unity," 2026, unity Technologies Documentation. [Online]. Available: https://unity.com/ecs